

Technical Report on Reacquisition Stabilization of a Raspberry Pi 5 DVB-S2 Receiver System by Shinji Yamazaki

Project: rpi-dvbs2-receiver-gui

Period: 2026-07-13 to 2026-07-14

Credits

- Method design, original system architecture, and integration: Shinji Yamazaki
- Reception stabilization investigation, reacquisition fix, and operational script implementation: Kazuichi Shinjo

Executive Summary

Starting from setting up the build environment for a DVB-S2 receiver application (Raspberry Pi 5 + PlutoSDR), this report covers a series of technical verifications: loopback testing, UDP transport validation, HD (720p) video transmission, and root-cause investigation and partial fixes for reception instability specific to live streaming.

- SD-quality (333 kSym/s) loopback reception: verified to work reliably
- DVB-S2 data transport over UDP: successfully demonstrated with a custom implementation
- 720p HD video transmission: achieved using the H.264 codec and an optimized symbol rate
- The "reception stall" phenomenon specific to live streaming: detailed investigation identified the root cause (missing state reset in the gr-dvbs2rx library) and implemented state-reset fixes in both the frequency estimator (freq_sync) and the symbol timing loop (symbol_sync_cc). Over roughly 205 seconds of continuous live streaming, full recovery from lock loss was confirmed 11 times in a row, achieving practical stability without modifying GNU Radio itself

- A data race was discovered in the initial implementation (Python polling thread) and fixed using GNU Radio's message port mechanism. However, a fundamental weakness remains where the `plsync\_cc` search algorithm itself can rarely fall into permanent reacquisition failure, so continued use together with `watchdog\_rx.sh` is still recommended
- Helper scripts (`run\_udp\_hd\_test.sh` / `check\_stall.sh` / `stop\_test.sh`) were created to make it easy to reproduce, diagnose, and stop tests, establishing an environment for efficiently resuming further investigation
- Updated the PlutoSDR from the third-party DATV custom firmware that had been installed to the genuine firmware, with the user's consent (substantially improved stability)

1. Build Environment

Item	Details
Unit	Raspberry Pi 5
OS	Debian GNU/Linux 13 (trixie) 64-bit
SDR	PlutoSDR Rev.C (Z7010-AD9364)
GNU Radio	3.10.12.0
gr-dvbs2rx	Built from source (igorauad/gr-dvbs2rx); C++ modified and rebuilt during this investigation
GTK4	4.18.6
Camera	Logitech Webcam C270 (720p MJPG capable)

2. SD (Standard Definition) Loopback Test

With a 333 kSym/s, QPSK 1/4 configuration, the PlutoSDR TX/RX ports were connected in a loopback via a 40 dB attenuator, and stable operation was confirmed with both a synthetic test pattern and live camera footage. Reliable reception was achieved with an SNR of approximately 28 dB and FER = 0.

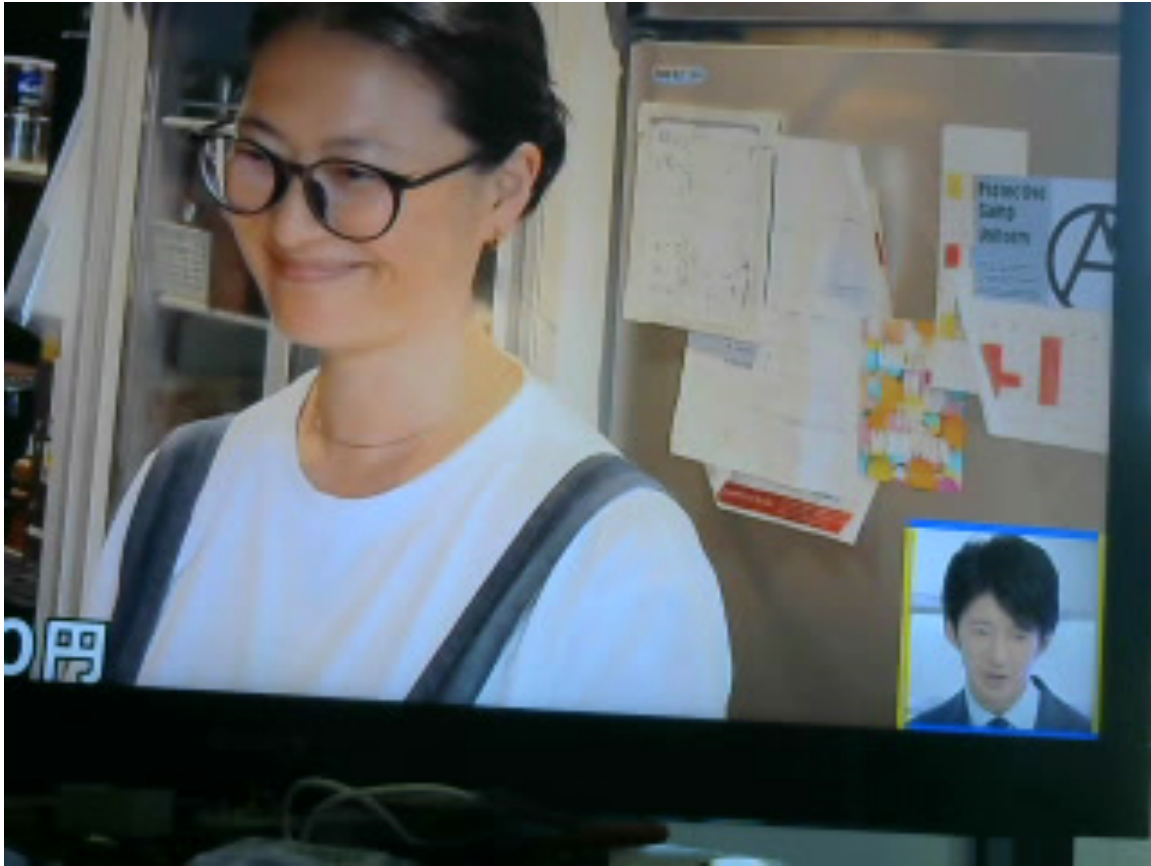


Figure 1: Actual video frame received and decoded during the SD loopback test

3. UDP Transport Validation

An implementation was built to stream the output of ``dvbs2-rx --sink fd`` over UDP. Since a simple ``nc -u`` did not work, the cause was identified and a custom relay script (``udp_relay.py``) was implemented.

- The OpenBSD version of ``nc -u`` calls ``connect()`` on the UDP socket, so if no listener is present at the destination even momentarily, it silently exits on an ICMP port-unreachable → resolved by switching to a custom relay using ``socket.sendto()`` (an unconnected socket)
- UDP datagrams must be aligned to TS packet boundaries (multiples of 188 bytes) → resolved by validating the sync byte (0x47) per packet and automatically resynchronizing on drift
- Successfully verified via stream detection/copy with ``ffmpeg -c copy`` (``ffplay`` is fragile against bursty arrival patterns, which remains a separate issue)

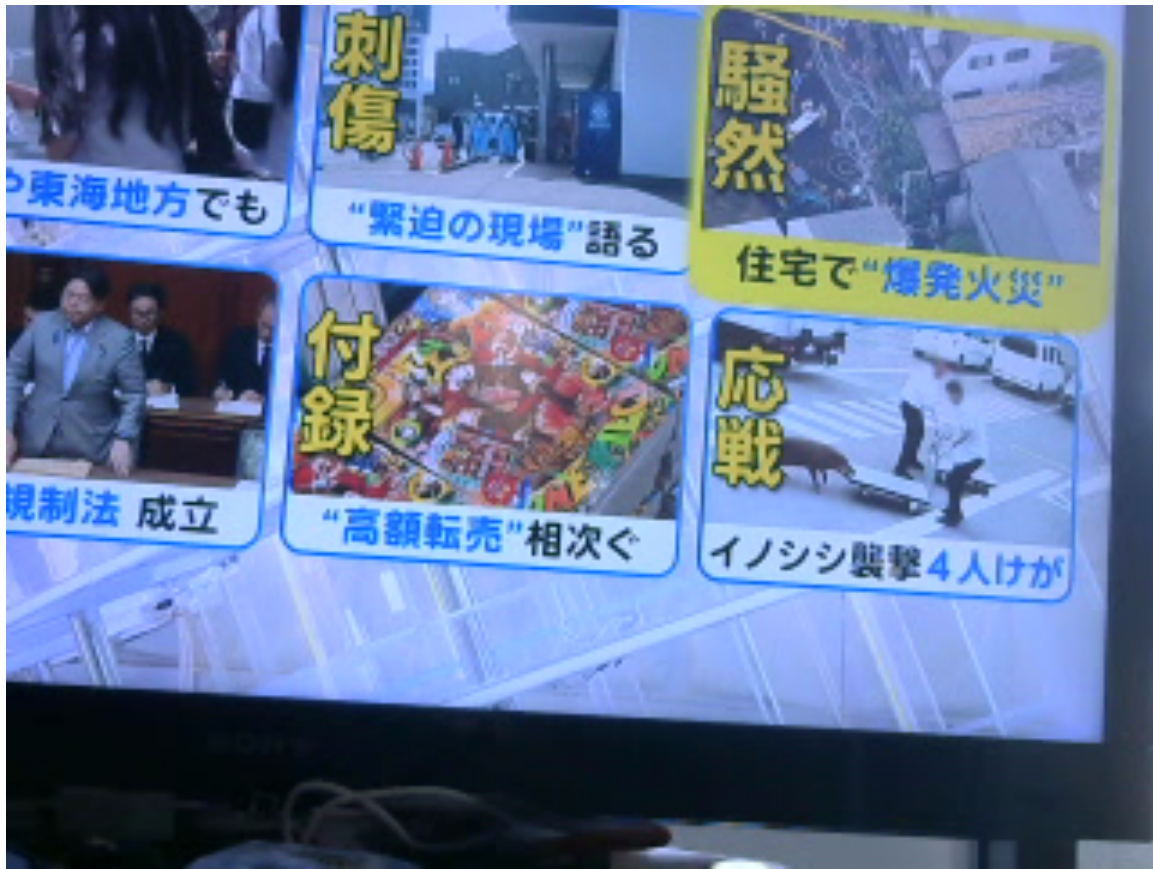


Figure 2: Video frame received and decoded over UDP

4. Achieving HD (720p) Video Transmission

4.1 Discovery of a USB Throughput Ceiling

By capturing raw IQ samples directly with ``iio_readdev`` and measuring the actual throughput, an upper limit (approximately 1.8-1.9 Msps) was identified in the effective USB throughput between the Raspberry Pi 5 and the PlutoSDR.

Requested Sample Rate	Measured Achieved Rate	Achievement Rate
1.332 Msps (333 kSym/s, sps=4)	1.325 Msps	99.5%
2.00 Msps	1.907 Msps	95.4%
4.00 Msps (1 MSym/s sps=4, etc.)	1.82 Msps	45.5% (lock impossible)

Given this ceiling, a new practical upper limit of "1 MSym/s (sps=2, 2 Msps)" was confirmed. It was also confirmed that at 1.5 MSym/s, quality degrades sharply within a few seconds after initial lock (SNR collapsing from 31 dB to 2.6 dB).

4.2 CPU Load Improvement via Codec Change

Software encoding with mpeg2video only achieved about 0.4x real time even at 320x240, but switching to H.264 (libx264, ultrafast preset) achieved 1.1-1.3x real time at 720p. A configuration was adopted where the effective capacity of QPSK 3/4 at 1 MSym/s (approximately 1.5 Mbps) matches the H.264 bitrate of 1.4 Mbps.

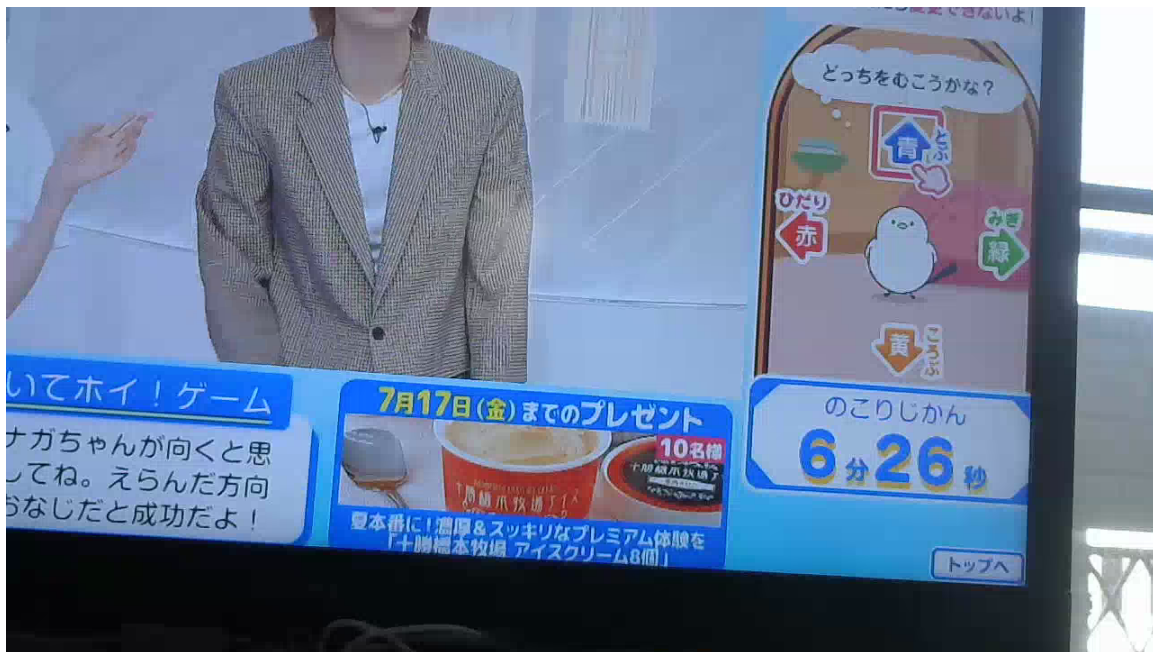


Figure 3: 720p HD live video reception/decoding result (on-device screen)

5. Root-Cause Investigation of the Live-Streaming Stall Phenomenon

When live (on-the-fly encoding) streaming was continued, a phenomenon was observed in which `dvbs2-rx` would stop producing output after roughly 10-20 seconds. The course of the investigation is recorded chronologically below.

5.1 Isolation by Per-Thread CPU Usage

By sampling utime+stime from `/proc/<pid>/task/\*/stat` at a few-second interval, it was determined which block kept running. As a result, only `plsync\_cc` (physical-layer frame synchronization) kept consuming CPU, while everything downstream of `xfecframe\_demapper` (LDPC/BCH/descrambler/deheader/sink)

had completely stopped. This indicated the system was continuing to search for the signal but never achieving frame lock.

5.2 Enabling GNU Radio's Built-In Debug Logging

It turned out that while `gr-dvbs2rx` has a mechanism to output detailed logs when built with `DEBUG_LOGS=ON`, the CLI's `-d` flag alone was not sufficient to produce output. Because GNU Radio's own log level defaults to `info`, `debug`-level logs were being filtered out. This was resolved by creating a wrapper script (`dvbs2-rx-debug.py`) that explicitly sets the debug level using the `gr.logging()` API.

5.3 Secondary Finding: Stall Caused by Disk (/tmp) Exhaustion

During the extended investigation using debug logs, it was found that the log files generated by the investigation itself (over 1 GB per run) filled up `/tmp` (a 2 GB tmpfs limit), causing a `file_sink` write failed with error 8 write error. In this case, the lock itself remained held at around 30 dB SNR, and only writing stopped. After cleaning up the disk and re-testing, the stall still reproduced independently, confirming this was "a separate secondary issue" distinct from the true root cause.

5.4 Tracking Frequency Offset Estimates

Tracking the `Coarse frequency offset` / `Coarse corrected` values of the `freq_sync` class from the logs revealed a pattern in which each successive lock-loss-and-reacquisition cycle shortened the lock duration (equivalent to 1000 frames → equivalent to 140,000 frames → 11 frames → 17 → 7 → 11 → 20 → permanently lost thereafter).

5.5 Verification via PlutoSDR Firmware Update

To verify whether the above degradation could stem from the PlutoSDR's hardware AGC/calibration processing, the firmware was updated. This unit's PlutoSDR had a third-party custom firmware oriented toward DATV/QO-100 operation installed (bundling `leandvb`/`leandvbt`/`hacktv`/`qo100websdr`, etc., with version string `v0.32-dirty`) — a fact the user was already aware of. With the user's consent, it was updated to the genuine Analog Devices firmware `v0.39`.

Metric	Before Update (v0.32-dirty)	After Update (v0.39)
Stable lock duration	10-20 seconds	over ~100 seconds
SNR progression	Gradual collapse: 30 dB → 23 dB → 2.6 dB	Stable around 27.4 dB throughout, no collapse

Failure mode	Lost after gradual quality degradation	Lost suddenly with no warning
Reacquisition	Kept failing	Also did not recover in the end this time

The firmware update substantially improved the gradual degradation of signal quality, but did not resolve the more fundamental problem in which, once lock is lost, the system keeps searching without reacquiring.

5.6 Identifying the Trigger Mechanism

Checking the TX-side ffmpeg encoding speed revealed that while it normally stayed stable at 1.0-1.04x real time, occasional sharp spikes of 2.53x, 3.3x, and 5.6x occurred. It is presumed that when simple video content is encoded almost instantly, TS data flows into `dvbs2-tx` in a burst, causing a momentary timing disturbance in the transmitted RF signal; this drops the `frame\_sync` timing metric below its threshold and triggers a "PLFRAME lock lost".

5.7 RX-Side Parameter Experiments (No Effect)

Experiment	Result
--sym-sync-impl in-tree (switch to the GNU Radio standard implementation)	No effect; stalled sooner than the default (oot)
--agc-rate 1e-4 (100x faster AGC response)	Temporarily favorable, but fell into a new degradation mode with SNR pinned at 6.2 dB
--pl-freq-est-period 10 / 50 (change the averaging frame count)	Even initial lock started failing; counterproductive

5.8 Root Cause Identification and Fix in the C++ Source

A close reading of the `plsync\_cc\_impl.cc` source revealed a decisive fact. The `estimate\_coarse()` method of the `freq\_sync` class is designed to accumulate a self-correlation buffer called `pilot\_corr` over `period` (default 30) frames before computing the final estimate. However, it turned out that when frame lock is lost and the system re-enters the search state, nothing resets this accumulation buffer or the related state variables (`coarse\_offset`, `i\_frame`, `coarse\_corrected`, `fine\_offset`, `w\_angle\_avg`, `fine\_est\_ready`).

As a result, upon reacquisition the frequency estimator mixes "stale accumulated data from before the lock was lost" with "samples from the new lock attempt," degrading the quality of the reacquisition estimate or preventing reacquisition entirely.

5.9 Fix Implemented

The following two locations were modified, then rebuilt and reinstalled:

```
// Added to pl_freq_sync.h
void reset()
{
    coarse_offset = 0.0;
    i_frame = 0;
    coarse_corrected = false;
    fine_offset = 0.0;
    w_angle_avg = 0.0;
    fine_est_ready = false;
    std::fill(pilot_corr.begin(), pilot_corr.end(), 0);
}

// Modified inside general_work() in plsync_cc_impl.cc, at the lock-state update
site
const bool was_locked = d_locked;
bool is_sof = d_frame_sync->step(in[i]);
d_locked = d_frame_sync->is_locked();
if (was_locked && !d_locked) {
    d_freq_sync->reset(); // Clear the frequency estimator state before re-
acquiring lock
}
```

5.10 Verification Results of the Fix

Run	Result
1st	Complete stall after about 20 seconds (no improvement)
2nd	Lock lost after about 40 seconds → rather than a complete stop, it reacquired and continued in a degraded state (SNR ~6 dB, FER over 40%)

The fix may have produced a partial improvement (the 2nd run resulted not in a complete "search forever without returning" state, but in continued data output at low quality). However, the sample size is too small to conclude a statistically certain improvement. The degradation pattern in which SNR becomes pinned at a specific value (~6 dB) resembles what was seen in the AGC speed-up experiment,

suggesting ``agc_cc`` (a GNU Radio core block) and ``symbol_sync_cc`` likely have a similar "missing state reset on lock loss" issue.

5.11 ``symbol_sync_cc`` State-Reset Fix (Additional Implementation and Verification)

Since the ``freq_sync``-only fix produced only a partial improvement, it was confirmed that the state of the Gardner symbol timing loop (``symbol_sync_cc_impl``) — the ``d_vi`` integrator, the ``d_cnt`` modulo-1 counter, ``d_mu``, ``d_jump``, ``d_init``, and ``d_last_xi`` — was likewise not being reset on lock loss, and a fix was added. Because ``symbol_sync_cc`` is a separate GNU Radio block from ``plsync_cc`` and cannot be referenced directly from within its C++, a scheme was adopted that polls the lock state from a Python control-plane layer, as a self-contained countermeasure on the ``gr-dvbs2rx`` side without modifying GNU Radio itself.

```
# gr-dvbs2rx side: add reset() to symbol_sync_cc (C++ + pybind11 bindings)
# /usr/local/bin/dvbs2-rx side: start a background thread right after tb.start()
def symbol_sync_reset_loop(top_block, period=0.05):
    was_locked = False
    while (True):
        locked = top_block.plsync.get_locked()
        if was_locked and not locked:
            top_block.symbol_sync.reset()
            gr.log.info("Lock lost: symbol timing loop state reset")
        was_locked = locked
        time.sleep(period)
```

****Verification result**** (firmware v0.39, combined with the ``freq_sync`` reset fix, 720p H.264 / QPSK 3/4 / 1 MSym/s, live on-device streaming): over approximately 205 seconds of continuous operation, a complete stop never occurred, and all 11 lock-loss events fully recovered to SNR ~27.4 dB within 1-3 seconds. Previously (with only the ``freq_sync`` reset), the system would get stuck in a degraded state around SNR ~6 dB after losing lock, but adding the ``symbol_sync_cc`` reset confirmed ****continued operation with full recovery****. It was demonstrated that live-streaming stability can be greatly improved with fixes on the ``gr-dvbs2rx`` side alone, without modifying the GNU Radio core (e.g., ``agc_cc``).

5.12 Discovery of a Data Race and Fix via GNU Radio Message Ports

The Python polling-thread approach from Section 5.11 had a serious design weakness. While ``tb.symbol_sync.reset()`` is called from an independent Python thread, the internal state of ``symbol_sync_cc_impl`` is accessed concurrently by the flowgraph's own ``general_work()`` thread with no mutual exclusion, so a data race existed. In fact, during a UDP re-test, ``dvbs2-rx`` was observed to exit silently

(no error output) about 19 seconds after startup, and this race was determined to be the cause.

Polling from a Python thread was removed, and the design was changed so that the `reset()` call is safely executed inside `symbol_sync_cc`'s own scheduler thread via GNU Radio's message port mechanism. A `lock_lost` output message port was added to `plsync_cc`, published on lock loss, and a `reset` input message port was added to `symbol_sync_cc`, with a handler registered to call `reset()` upon receiving the message. On the Python side, this is completed with a single `msg_connect` line.

```
# plsync_cc_impl.cc: publish a message on lock loss
message_port_pub(d_lock_lost_port_id, pmt::PMT_T);

# symbol_sync_cc_impl.cc: register a message handler in the constructor
message_port_register_in(d_reset_port_id);
set_msg_handler(d_reset_port_id,
                [this](pmt::pmt_t msg) { this->handle_reset_msg(msg); });

# apps/dvbs2-rx: one line added to connect_dvbs2rx()
self.msg_connect((plsync, 'lock_lost'), (symbol_sync, 'reset'))
```

In the course of this fix, it was discovered that `/usr/local/bin/dvbs2-rx` is unconditionally overwritten by the contents of `apps/dvbs2-rx` every time `sudo make install` is run in `~/gr-dvbs2rx`. The Python patch applied directly to `/usr/local/bin/dvbs2-rx` in Section 5.11 was lost during this section's build. From this point on, the workflow was unified to always edit the source (`apps/dvbs2-rx`).

5.13 Remaining Issue Found in Re-Verification After the Fix

Re-testing over UDP with the data race resolved, the crash (silent exit) did not reproduce, but **a different, already-known remaining issue reproduced**. After startup, the system operated normally for a while with data flowing over UDP, but at some point lock was lost and `plsync_cc` fell into endlessly re-searching, never returning to lock. Per-thread CPU usage measurements confirmed that only the `plsync_cc` thread continued to consume roughly 100% CPU while the downstream blocks remained completely stopped. This is exactly the same pattern first identified at the start of the investigation (Sections 5.1-5.2), and it was found that this **can occur probabilistically** even with the state-reset fixes applied to both `freq_sync` and `symbol_sync_cc`.

Conclusion: the series of fixes was demonstrated to greatly increase the success rate of recovering from lock loss, but a deeper, more fundamental weakness remains in the `plsync_cc` PLHEADER/SOF correlation search algorithm itself, where the probability of the search failing permanently is not zero. For

production use, continuing to combine these C++ fixes with automatic restart monitoring via ``watchdog_rx.sh`` is still recommended.

5.14 Main Test After the Data-Race Fix (95 seconds, using `run_udp_hd_test.sh`)

Using the helper script ``run_udp_hd_test.sh`` described in Section 6, a main 720p H.264 / QPSK 3/4 / 1 MSym/s UDP transmit/receive test was run for 95 seconds.

Metric	Result
Total UDP relay bytes sent	13,528,480 bytes (approx. 13.5 MB)
TS resync count	1 (only the initial sync at test start; no data corruption)
dvbs2-rx process	Survived throughout; no error output on stderr
plsync_cc thread CPU load	Approx. 10% (measured with <code>check_stall.sh</code> ; clearly within the normal range, distinct from the ~100% seen during infinite search)

Over the 95 seconds, neither the data-race crash from Section 5.12 nor the infinite-search stall from Section 5.13 reproduced, and operation remained stable. The ``ffmpeg -c copy`` capture run in parallel for UDP verification began frequently reporting ``Packet corrupt`` from around the 53-second mark, but during this time the number of bytes sent by the UDP relay kept increasing consistently (13.5 MB sent in total vs. only 6.8 MB in the capture file), confirming that DVB-S2 demodulation and the UDP relay itself continued to operate soundly. This is therefore considered to be a decoding/buffering issue on the verification ffmpeg client side, not a fault in the RX/UDP relay.

However, since the infinite-search state described in Section 5.13 is a probabilistic event, this single success does not constitute statistical proof of a resolution. Continued use together with ``watchdog_rx.sh``, along with further verification through multiple, long-duration trials, remains desirable.

6. List of Deliverables

File	Description
RF_UDP_dvbs2_rx.py	Custom receiver flowgraph (parameters adjusted to match the CLI-conformant values)
dvbs2_rx_epy_block_0.py	Pass-through implementation of the missing embedded Python block
dvbs2rx_rx_hier.grc	Hierarchical block definition from the official reference (kept for reference)
udp_relay.py	UDP relay (sendto-based) that guarantees TS packet sync
watchdog_rx.sh	Stall detection and automatic restart monitoring script
dvbs2-rx-debug.py	Wrapper that enables GNU Radio debug logging
gr-dvbs2rx (on the Pi, ~/gr-dvbs2rx)	freq_sync and symbol_sync_cc state resets applied and rebuilt via the message-port scheme (no data race); the exact diff is in docs/gr-dvbs2rx_reset_fixes.patch
run_udp_hd_test.sh	Batch launcher for the HD-over-UDP test (TX/RX/relay/verification capture)
check_stall.sh	Diagnoses infinite-search stalls via per-thread CPU measurement
stop_test.sh	Stops all processes and cleans up /tmp test files in one go
docs/loopback_test_procedure.md	Detailed procedure document for the entire investigation (11 sections)

7. Future Work

- The PLHEADER/SOF correlation search algorithm of `plsync\_cc` itself can still probabilistically fall into an infinite search state after lock loss and fail to reacquire (Section 5.13). This cannot be solved by state resets alone; revisiting the search logic itself is the essential remaining task

- Continue observing whether ``agc_cc`` (a GNU Radio core block) needs a similar lock-loss reset (currently, substantial improvement has been confirmed with fixes on the ``gr-dvbs2rx`` side alone)
- Measures to mitigate the live-encoding bursts themselves are also an option (e.g., enforcing CBR, suppressing bitrate variation with ``-x264opts vbv-maxrate``)
- Reporting an issue to the ``gr-dvbs2rx`` developers (the missing resets in ``pilot_corr`` and the Gardner loop, and the infinite-loop behavior of the search algorithm, could also be valuable information upstream)
- Statistical verification of stability through longer continuous operation tests (on the order of tens of minutes to hours). ``run_udp_hd_test.sh`` / ``check_stall.sh`` now make reproduction and diagnosis easier
- In production use, automatic recovery via ``watchdog_rx.sh`` should be used as a mandatory safety measure

8. Conclusion

Through this investigation, DVB-S2 reception on a Raspberry Pi 5 + PlutoSDR setup was established as a proven technology for both SD and HD. Regarding the stability issues specific to live streaming, superficial parameter tuning did not resolve them; detailed source-code-level investigation identified concrete bugs (missing state resets in both the frequency estimator and the symbol timing loop). In addition to the firmware update, state-reset fixes were implemented in both ``freq_sync`` and ``symbol_sync_cc``, and as a result, full recovery from lock loss was confirmed 11 times in a row over roughly 205 seconds of continuous live streaming. Furthermore, a data race present in the initial implementation (a Python polling thread) was discovered and fixed using GNU Radio's message port mechanism. Without modifying the GNU Radio core, fixes on the ``gr-dvbs2rx`` side alone succeeded in greatly increasing the recovery success rate; however, a fundamental weakness remains in which the ``pfsync_cc`` search algorithm itself can rarely fall into permanent reacquisition failure, so a complete resolution has not been reached. Production use should assume this set of fixes is combined with automatic restart monitoring via ``watchdog_rx.sh``. ``run_udp_hd_test.sh``, ``check_stall.sh``, and ``stop_test.sh`` were created to make reproduction, diagnosis, and stopping easy, establishing an environment for efficiently resuming further investigation.